

BOS2L-DP: Certified Recovery for Overlap Scheduling under Monotone Delays

Chen-Hong

Department of Computer Science and Information Engineering
National Central University, Taoyuan, Taiwan
leon28473787@gmail.com

Abstract—Material unavailability and temporary machine outages can invalidate an overlap-aware production schedule after execution has begun. This paper develops a two-layer recovery formulation: a BOS list policy produces the executed prefix, and a progress-state dynamic program optimizes its continuation after a disturbance. We distinguish monotonicity of the optimal completion time from the potentially nonmonotone output of a greedy policy. For the same prefix and operational rules, removing feasible continuations cannot improve the optimum. If a new disturbance disappears within D slots and the reference continuation has no future restrictions, its unavoidable completion-time penalty lies in $[0, D]$. We derive explicit lower and upper certificates and an interval for this penalty. An earliest-arrival dynamic program is exact for an integer-slot model with free preemption and migration, progress-triggered overlap, known job blackouts, and aggregate machine availability. It is exponential in the number of unfinished tasks, not an FPTAS. On 384 small synthetic instances, all optima are certified, including 339 cases closed without state expansion; 24 independent MILP checks agree. A reproducible original-BOS counterexample improves from 12 to 11 after a release delay, while the two continuation optima both equal 11. These results support certified small-instance recovery, not a general $(2 - 1/m + \varepsilon)$ approximation theorem or an online guarantee.

Index Terms—overlap scheduling, rescheduling, dynamic programming, progress state, disturbance recovery, lower and upper bounds

I. INTRODUCTION

An overlap-aware schedule can start a successor after a sufficient prefix of its predecessor has been processed. This creates flexibility, but a material interruption or an unavailable machine can invalidate the planned handoff. The relevant operational question is then not simply whether a new makespan is larger. It is how much deterioration is unavoidable, and how much additional loss is attributable to the recovery policy.

The September 15 BOS draft [1] studies identical machines, nonpreemptive tasks, and start-start lags. Its BOS priority is an overlap-adjusted bottom level. Its zero-release analysis provides the list-scheduling bound

$$C_{\text{BOS}} \leq W/m + (1 - 1/m)L_\delta \leq (2 - 1/m) \text{OPT}_{\text{np}}. \quad (1)$$

Here W is total work and L_δ is a lag-adjusted longest path. This is a work-conserving list-policy bound, not a tight characterization of the BOS priority. The draft separately gives a factor-two bound for arbitrary releases. Neither statement automatically transfers to preemptive repair, progress-based constraints, or machine calendars.

Our starting observation is a partial order on *instances*: when a disturbance only removes feasible continuations, the best completion time cannot decrease. This is separate from the directed progress transitions used to solve each instance. In particular, there is no requirement that a worse environment produce a worse greedy output. Timing anomalies in list scheduling have a long history [2]; we give a witness for the actual September 15 BOS implementation.

The contribution of this draft is a precise, reproducible combination of (i) a fixed-prefix progress model, (ii) monotonicity and additive-delay certificates, (iii) an exact finite-state recurrence with waiting compression, and (iv) independently checked small-instance experiments. The constituent ideas of dynamic programming and resource-constrained scheduling are established [3]–[5]. We do not claim that monotonicity, Bellman’s principle, or rescheduling itself is new. A stronger structural result and realistic benchmarks are still needed before making a broad novelty or industrial-performance claim.

II. A FIXED-PREFIX PROGRESS MODEL

A. Operational state and scope

Let $G = (V, E)$ be a DAG with n tasks and m identical unit-speed machines. Processing requirements p_i are positive integers. One slot is the chosen physical processing quantum; all releases and calendar endpoints are integer slot boundaries. A task uses at most one machine in a slot. It may be preempted or migrated between slots at zero cost, but cannot execute on two machines simultaneously.

At the disturbance time τ , execution before τ is frozen. The vector q^0 records completed processing, with $0 \leq q_i^0 \leq p_i$. Both the reference and disturbed continuations use exactly this prefix and the same preemption/migration rules. Previous machine identities need not be retained because machines are identical and migration is free. A model with affinity, setup, restart loss, or migration cost would require additional state and is outside our claims.

Task i has release r_i and a finite list of blackout intervals. Write $g_i(t) \in \{0, 1\}$ for whether it may execute in slot $[t, t + 1)$. Aggregate capacity $a(t) \in \{0, \dots, m\}$ specifies the number of available machines. All gates and capacities are known for the repair problem and eventually become $g_i(t) = 1$, $a(t) = m$. Overlapping outage records specify absolute available capacities and combine by minimum, not

by summing machine losses. These are deterministic recovery instances, not a policy for unrevealed future events.

B. Overlap depends on actual progress

An edge (u, v) has threshold $\ell_{uv} \in \{0, \dots, p_u\}$. When v first executes, a positive threshold requires

$$q_u(t) \geq \ell_{uv}. \quad (2)$$

For $\ell_{uv} = 0$, u must have started previously or start in the same slot. This preserves the original DAG's zero-lag start order; simultaneous starts are resolved in topological order. Once v has started, its incoming conditions remain satisfied. All incoming edges have AND semantics: this is not arbitrary subset-kit activation. Job gates are exogenous availability restrictions, not a detailed bill-of-materials consumption model.

Without preemption, $q_u(t) \geq \ell_{uv} > 0$ at a successor start is equivalent to $s_v \geq s_u + \ell_{uv}$ for an integer-slot schedule. With preemption, replacing progress by wall-clock elapsed time is incorrect. For example, a task of length 3 executes in slot 0 and is blocked in slots 1–3. Its threshold of 2 is reached at time 5, not time 2. A one-slot successor then finishes at time 6. This case is a regression test in the supplied implementation.

C. Actions and objective

Let q be the progress vector at a slot boundary. An action $A \subseteq V$ contains unfinished tasks to process for one slot, with

$$|A| \leq a(t), \quad g_i(t) = 1 \quad (i \in A), \quad (3)$$

$$q'_i = q_i + \mathbf{1}\{i \in A\}. \quad (4)$$

New starts satisfy (2) and the zero-threshold condition. Denote the resulting legal actions by $\mathcal{A}_\theta(t, q)$, where θ contains releases, gates, and capacity. The empty action allows waiting. The objective is the *absolute* completion time

$$C = \min\{t \geq \tau : q(t) = p\}. \quad (5)$$

Every bound below uses this same origin. The equivalent remaining duration is $C - \tau$, but the two quantities are never mixed in a ratio. Write $\text{OPT}_\theta(q^0, \tau)$ for the optimum under this fixed history. It is not the optimum obtained by rescheduling the past.

III. MONOTONE DISTURBANCES AND NEW BOUNDS

A. Instance order versus progress order

For the same (G, p, q^0, τ, m) , write $\theta \preceq \theta'$ when $g'_i(t) \leq g_i(t)$ and $a'(t) \leq a(t)$ at every future slot. Increasing a release is represented by removing available slots. The annotated instance may be called G' , but the task-edge topology need not change.

Theorem 1 (Fixed-prefix monotonicity). *If $\theta \preceq \theta'$, then*

$$\mathcal{F}_{\theta'} \subseteq \mathcal{F}_\theta, \quad \text{OPT}_\theta \leq \text{OPT}_{\theta'}. \quad (6)$$

Proof. Take any feasible continuation for θ' . Replay exactly the same actions at the same absolute times in θ . Every job slot remains available, capacity is no smaller, and the identical progress sequence satisfies the same handoff conditions. Thus

the continuation is feasible for θ , proving inclusion. Minimizing the same completion objective over these sets proves the inequality. $\square$

The result concerns optima, not outputs of a particular priority rule. It also concerns absolute completion time from a common prefix; an unconditional claim about the remaining duration as time advances would be false. Removing restrictions or changing permitted controls breaks the premise.

B. A finite-disturbance additive bound

For the following theorem, the reference has no future restrictions: $g_i(t) = 1$ and $a(t) = m$ for all $t \geq \tau$. The disturbed instance may have arbitrary additional job gates and capacity reductions, but all of them end by $B = \tau + D$. Assume there is unfinished work.

Theorem 2 (Shift bound). *Under these conditions,*

$$0 \leq \Delta^* := \text{OPT}_d - \text{OPT}_0 \leq D. \quad (7)$$

For any feasible reference continuation of completion U_0 , shifting all its future actions by D is a disturbed continuation of completion $U_0 + D$.

Proof. The lower inequality follows from (6). For the upper inequality, keep the prefix fixed, idle from τ to $\tau + D$, and replay an optimal reference continuation shifted by D . All future actions occur after the disturbance has disappeared. The action order and progress preceding each new start are unchanged, so every threshold and capacity constraint holds. The completion is $\text{OPT}_0 + D$. The same construction applies to any feasible reference continuation. $\square$

This bound can be tight: a single remaining task blocked throughout $[\tau, \tau + D)$ loses exactly D . It can also be zero when slack or parallel work absorbs the disturbance. D is a bound on the end of the new disturbance relative to τ , not the sum of all task delays. The shift argument is not asserted for a reference with arbitrary future machine calendars: shifting could collide with a later outage.

C. Computable lower bounds

Let $w_i = p_i - q_i^0$ and $W_R = \sum_i w_i$. Work and per-task bounds are

$$L_W = \tau + \lceil W_R/m \rceil, \quad L_J = \tau + \max_i w_i. \quad (8)$$

Available capacity gives

$$L_A = \min \left\{ T \geq \tau : \sum_{t=\tau}^{T-1} a(t) \geq W_R \right\}. \quad (9)$$

For a stronger path-and-gate relaxation, remove the machine limit but retain one-unit-per-task-per-slot speed, all releases, gates, and thresholds. Process every eligible task in each slot, closing zero-lag starts topologically. Denote its completion by L_∞ .

Lemma 3 (Unlimited-machine relaxation). *The stated unlimited-machine schedule is optimal for its relaxation, and $L_\infty \leq \text{OPT}_d$.*

Proof. Induct on slots. Suppose its progress dominates that of another relaxed feasible schedule coordinatewise. Each positive threshold that the latter satisfies is satisfied by the former. For zero thresholds, follow a topological order: a predecessor already started or selected in the competing action is already progressed, completed, or also selectable by the closure. Thus every competing increment can be matched unless the dominating coordinate is already larger. Job gates are the same and there is no shared capacity limit. Dominance is preserved, proving earliest completion. Removing capacity constraints cannot increase the optimum of the original problem. $\square$

An old valid bound $L_0 \leq \text{OPT}_0$ remains valid under monotone disturbances. Therefore define

$$L_d = \max\{L_0, L_W, L_J, L_A, L_\infty\}. \quad (10)$$

These terms are maximized, not added, to avoid double counting.

D. Upper bounds and the requested certificate

Let C_{BP}, C_{BC}, C_{RL} be feasible repair schedules defined in Section V. Under the shift theorem's conditions,

$$U_d = \min\{C_{BP}, C_{BC}, C_{RL}, U_0 + D, B + W_R\}. \quad (11)$$

The last candidate waits to B and processes remaining tasks serially in a topological order. More generally it remains valid with B equal to the last release or calendar endpoint, even without a clean reference. The shifted-reference term is used only when its hypotheses hold. All upper bounds are backed by feasible schedules.

The main formula is the bracket

$$L_d \leq \text{OPT}_d = C_{DP} \leq U_d, \quad (12)$$

where equality with C_{DP} requires a completed exact DP solve. It is complemented by a directly computable loss interval:

$$\max\{0, L_d - U_0\} \leq \Delta^* \leq \min\{D, U_d - L_0\}. \quad (13)$$

Indeed, subtract $\text{OPT}_0 \leq U_0$ from $\text{OPT}_d \geq L_d$ for the lower endpoint, and subtract $\text{OPT}_0 \geq L_0$ from $\text{OPT}_d \leq U_d$ for the upper endpoint, then intersect with (7).

For any returned incumbent U and positive lower certificate L ,

$$0 \leq U - \text{OPT}_d \leq U - L, \quad 1 \leq U / \text{OPT}_d \leq U / L. \quad (14)$$

This is an instance certificate, not a uniform approximation constant. If a separate proof established $U_0 \leq \rho_0 \text{OPT}_0$, shifting would give $U_d / \text{OPT}_0 \leq \rho_0 + D / L_0$. The normalized delay D / L_0 is instance-dependent and is not an arbitrarily selectable accuracy parameter. In particular, (1) cannot be substituted for ρ_0 without reconciling its nonpreemptive, whole-instance optimum with the preemptive, fixed-prefix optimum used here.

IV. EXACT PROGRESS-STATE DYNAMIC PROGRAMMING

A. Bellman recurrence

For integer t and reachable q , let $V_\theta(t, q)$ be the earliest absolute completion of a continuation from that state. The basic recurrence is

$$\begin{aligned} V_\theta(t, p) &= t, \\ V_\theta(t, q) &= \min_{A \in \mathcal{A}_\theta(t, q)} V_\theta(t + 1, q + \mathbf{1}_A). \end{aligned} \quad (15)$$

The empty action explicitly permits waiting. A finite feasible upper schedule bounds the horizon; nonterminal states at the horizon have value $+\infty$. Exhaustive evaluation is exact by the usual decomposition of a feasible schedule into its first slot and a remaining continuation [3].

B. Compressing time with earliest-arrival labels

The implementation uses $F(q)$, the earliest boundary at which q can be reached, initialized by $F(q^0) = \tau$. Ignore calendars temporarily and let $\mathcal{A}(q)$ contain all nonempty task subsets of size at most m satisfying unfinished-work and progress/zero-lag start conditions. For $A \in \mathcal{A}(q)$ define

$$N_A(t) = \min\{s \geq t : |A| \leq a(s), g_i(s) = 1 \ (i \in A)\}. \quad (16)$$

Each action supplies the relaxation

$$F(q + \mathbf{1}_A) \leftarrow \min\{F(q + \mathbf{1}_A), N_A(F(q)) + 1\}. \quad (17)$$

Since $\sum_i q_i$ strictly increases, these transitions form an acyclic *state graph*. This graph is distinct from the task DAG. An edge can include idle time before its one-slot action.

Theorem 4 (Exactness of waiting compression). *The recurrence (17), over all structurally legal nonempty actions, gives $F(p) = \text{OPT}_\theta(q^0, \tau)$.*

Proof. For a fixed progress vector, an earlier arrival can imitate any later arrival by idling. No omitted history affects feasibility under the stated free-migration, zero-cost model. Further, $N_A(t)$ is nondecreasing in t . Consider the last nonempty slot of any feasible path to a state q' . It uses some action A from $q = q' - \mathbf{1}_A$. By induction in total progress, $F(q)$ is no later than that path's arrival at q ; thus $N_A(F(q)) + 1$ is no later than the considered arrival at q' . Conversely, every relaxed label is realized by waiting until $N_A(F(q))$ and executing the legal action. The two directions prove equality. $\square$

Nonmaximal subsets are included. The proof does not assume that an optimal policy must fill all available machines at every time.

C. Pruning, budgets, and complexity

An admissible absolute estimate for a label (t, q) is

$$\begin{aligned} h(t, q) &= t + \max\left\{\left\lceil \frac{\sum_i (p_i - q_i)}{m} \right\rceil, \max_i (p_i - q_i)\right\}, \\ H(t, q) &= \max\{L_d, h(t, q)\}. \end{aligned} \quad (18)$$

The code evaluates labels in increasing H , reopens a state when an earlier arrival is found, and prunes labels with $H \geq$

U , where U is a feasible incumbent. Parent actions recover a checkable schedule. The implementation’s initial bound omits inherited L_0 if none is supplied; adding a valid inherited bound only strengthens the certificate.

If a budget expires before expansion, the current label is returned to the frontier. With stale labels excluded, a valid lower certificate is

$$L_{\text{front}} = \min(\{U\} \cup \{H(t, q) : (t, q) \text{ on frontier}\}). \quad (19)$$

Every not-yet-eliminated better solution has a surviving frontier ancestor; admissibility and earlier-arrival dominance justify the bound. An empty frontier or $L_{\text{front}} = U$ certifies optimality. Budget-limited outputs retain $L_{\text{front}} \leq \text{OPT} \leq U$ and are never labeled exact unless the bounds meet.

The progress box has at most

$$N = \prod_{i=1}^n (w_i + 1), \quad K = \sum_{k=1}^{\min(m, n)} \binom{n}{k} \quad (20)$$

states and candidate subsets per state, respectively. A topological evaluation of (17) needs $O(N)$ label storage and $O(NK)$ transition relaxations, excluding action legality and calendar search. A naive slot scan adds a factor bounded by the finite horizon and the cost of checking gates. The A*-ordered implementation may reopen labels; its expansion count is therefore reported separately and is not claimed to equal the number of states. The product bound is exponential in task count and depends on numerical processing sizes. This is neither a polynomial-time exact algorithm nor an FPTAS. Exactness is for the chosen integer-slot model; no discretization-error bound to an arbitrary continuous-time model is claimed.

V. REPRODUCIBLE EXPERIMENTS

A. Design and fair comparisons

All data in this section were generated for this revision. The old September 15 implementation is included unchanged to produce the first-layer schedule and the timing-anomaly witness. We generate three families: sparse ordered DAGs (edge probability 0.20), two-node-layer DAGs (cross-layer probability 0.38), and fork-join DAGs with one source and one sink. Processing times are uniform integers in $[1, 4]$; each edge threshold is uniform in $[0, p_u]$. Zero-lag edges are included.

The design is 3 families $\times 2$ sizes ($n = 6, 8$) $\times 2$ machine counts ($m = 2, 3$) $\times 8$ seeds $\times 4$ severities ($D = 0, 1, 3, 6$), totaling 384 cases. Seed values and exact input graphs are stored. A nonpreemptive BOS schedule determines $\tau = \max(1, \lfloor C_{\text{BOS}}/3 \rfloor)$ and its progress prefix. Each base instance is held fixed across severities. Seed index modulo three selects a material gate, a reduction of capacity to $m - 1$, or both, lasting $[\tau, \tau + D)$. The material victim is a deterministic unfinished task. These are paired nested disturbances, not independent draws of 384 unrelated graphs; severities are equally represented but the three disturbance modes have 3/3/2 seeds per cell.

All repair policies share the same progress semantics, prefix, calendars, free preemption/migration, and feasibility checker.

TABLE I
DISTURBED-CASE GAPS TO EXACT FIXED-PREFIX OPTIMA (288 CASES).

Policy	Mean gap (%)	Max gap (%)	Optimal hits
BOS-P	2.47	25.00	232/288
BOS-C	2.70	25.00	227/288
R-LPT	2.08	28.57	244/288
Portfolio	1.30	25.00	260/288

BOS-P recomputes the selected set each slot using the original static BOS scores; BOS-C first retains eligible tasks from the preceding slot and then fills by BOS score. At the first recovery slot it has no retention preference. R-LPT selects largest remaining processing time. Ties use task index. These are explicitly named repair variants, not claims that the original nonpreemptive BOS code implements preemption. The portfolio takes the best of these three schedules and a shifted *heuristic* reference schedule. It does not use the reference DP optimum as its candidate.

The DP has a 250,000-expansion and 15-second per-case budget. A separate time-indexed MILP, implemented with SciPy/HiGHS [6], [7], checks 24 cases: $n = 6$, the first two seeds, and severities 0 and 3, across families and machine counts. The MILP has binary per-slot processing and first-start variables; Appendix A gives the constraints. It shares input data, not DP transitions or dominance pruning. An additional 90 random four-task instances are checked against explicit time-layer enumeration including idle actions. Both validation methods test the same integer-slot model.

B. Results and their limits

All 384 cases are certified optimal; all 24 MILP comparisons terminate optimally and agree. No replay-feasibility, monotonicity, shift-bound, or bracket violation is observed. The analytic bound is tight in 375/384 cases, and 339/384 cases require no DP expansion because an initial feasible schedule meets its lower bound. The maximum stored-label count is 273. The measured mean and maximum DP-call times are 0.471 and 4.897 milliseconds on the recorded environment; they include candidate schedules and bound construction but exclude MILP and plotting. These are single-run diagnostics on easy, small synthetic instances, not robust performance estimates.

Among 288 disturbed cases, 112 have zero unavoidable delay. Figure 1 shows mean loss versus severity and the heuristic gaps. Table I reports all disturbed cases, including ties. R-LPT has a smaller mean gap than BOS-P on this set; there is no claim of BOS dominance. DP provides exact recovery at the cost of potential state explosion. A separate 16-case check with $n = 8, 10, 12, 14$ certifies all cases, but 15 close without expansion. That result does not establish large-instance scalability even though one theoretical state box exceeds 34 million states.

A separate barrier stress test exposes a less favorable outcome. Three complete-precedence layers each contain w length-two tasks, with $m = 3$ and capacity two only in the

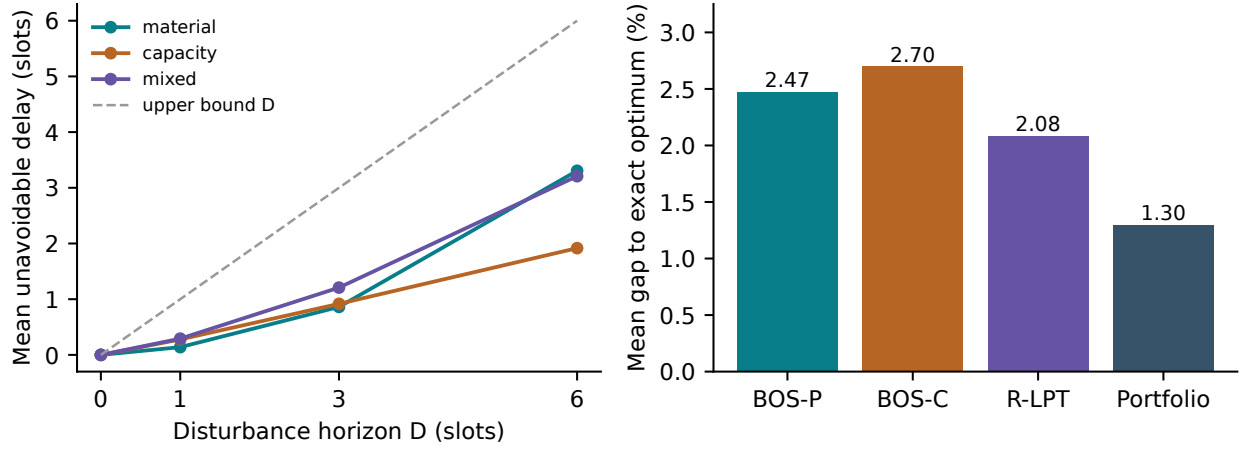


Fig. 1. Paired synthetic experiments. Left: exact unavoidable delay is nondecreasing with disturbance severity and lies below D . Right: mean gap of feasible repair candidates to the disturbed fixed-prefix optimum. No confidence interval or population-level superiority claim is made.

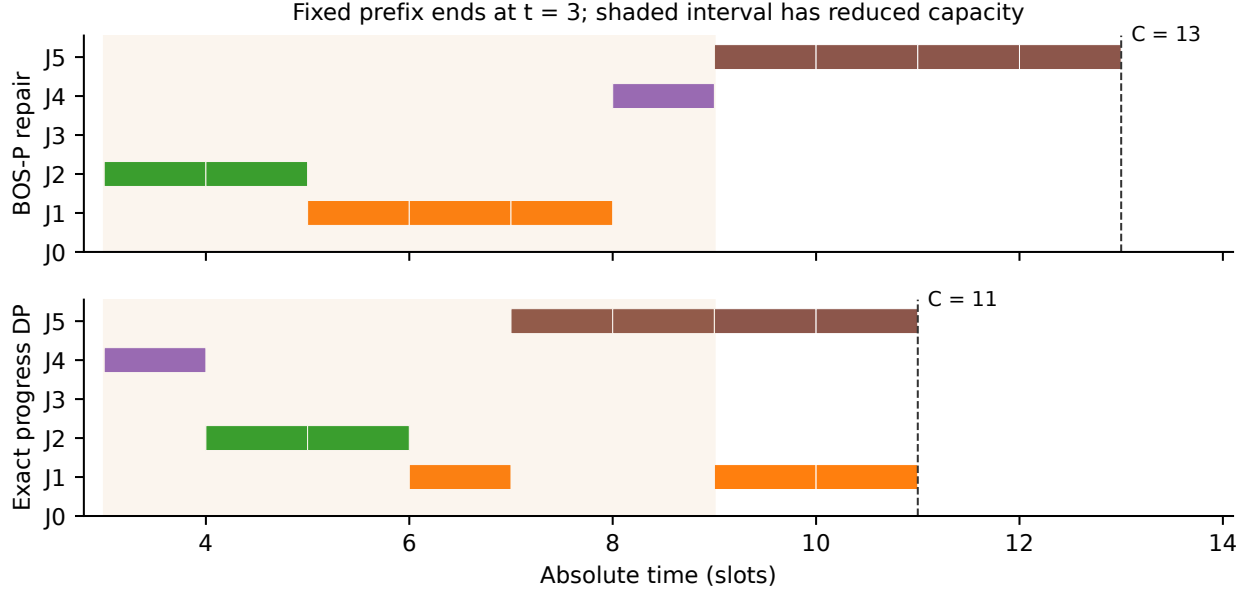


Fig. 2. A capacity-disturbance recovery example. Each row is a task; each colored unit is one processing slot. The exact DP reaches the capacity lower bound of 11, compared with BOS-P completion 13.

first slot. For $w = 4, 6, 8$, the layer-wise capacity bounds sum to 9, 13, 18, respectively; returned feasible schedules attain these values, independently establishing their optima. The general solver closes the first two cases immediately. For $w = 8$ ($n = 24$), its two-second budget expires with the valid certificate $17 \leq \text{OPT} \leq 18$, even though the specialized layer argument proves 18. The saved run expands 7,339 labels; budget-dependent counts vary across reruns. This demonstrates both the computational limitation and the value of adding structural lower bounds. These three constructed cases are excluded from Table I.

C. A worked recovery instance

For the stored example, $m = 2$, $\tau = 3$, $p = (3, 3, 2, 2, 1, 4)$ and $q^0 = (3, 0, 0, 2, 0, 0)$. Edges (u, v, ℓ) are $(0, 1, 2)$, $(0, 2, 2)$, $(0, 3, 1)$, $(0, 4, 1)$, $(1, 5, 1)$, $(2, 5, 2)$, $(3, 5, 1)$, and $(4, 5, 0)$. Capacity drops to one machine on $[3, 9)$, so $D = 6$ and $W_R = 10$. The capacity lower bound is $L_A = 11$: six processing units fit before time 9 and the remaining four require two slots at full capacity. The reference optimum is 9; disturbed BOS-P finishes at 13, while DP finishes at 11. Thus the unavoidable penalty is 2, and BOS-P adds two more slots of recoverable policy loss. Figure 2 displays the two continuations. The first-layer prefix is identical.

D. Original BOS need not worsen after a delay

The stored witness uses $m = 2$, $p = (4, 3, 2, 1, 3, 2, 3, 3)$ and edges $(0, 4, 1)$, $(1, 5, 2)$, $(1, 7, 2)$, $(2, 5, 1)$, $(2, 6, 1)$, $(3, 4, 0)$, $(3, 5, 0)$, $(3, 7, 1)$, and $(5, 6, 1)$. At $\tau = 4$, both executions have $q^0 = (2, 3, 2, 1, 0, 0, 0, 0)$. Originally all releases are zero. Increasing only task 5's release to 5 changes the original, nonpreemptive BOS completion from 12 to 11. The original start vector is $(2, 0, 0, 3, 6, 4, 6, 9)$; the delayed one is $(2, 0, 0, 3, 4, 6, 7, 8)$. The prefix before time 4 is unchanged. The input, machine assignments, and outputs are retained in JSON.

The progress-DP optima of the reference and delayed continuations are both 11, allowing the same preemption rules in both branches. Therefore the optimal-value monotonicity theorem is intact, while the stronger assertion $C_{\text{BOS},d} \geq C_{\text{BOS},0}$ is false. The perturbation changes a locally chosen ordering and removes an avoidable idle tail. This witness was found by a separate seed search and is excluded from the aggregate experiment statistics.

VI. DISCUSSION AND NEXT RESEARCH QUESTIONS

The state viewpoint is useful for two distinct reasons. The instance order permits inherited lower certificates, while the progress order permits exact dynamic programming and earlier-arrival dominance. Monotonicity alone does not imply a small state space, nor a useful constant-factor bound for an arbitrary recovery policy.

The appropriate current two-layer claim is conditional: given the prefix produced by first-layer BOS, the second layer returns an exact continuation for a completed grid-DP solve, or a feasible continuation with a certified interval if the budget is exhausted. This does not make the entire history globally optimal. Repeated unforeseen events require new continuations and an explicitly chosen online or robust benchmark. They cannot be justified by silently comparing to a clairvoyant whole-instance optimum.

Three limitations are immediate. First, a real process may permit interruptions only at designated checkpoints or require restart/setup costs; progress alone is then insufficient. Second, the deterministic model assumes the disruption duration is known when solving. Third, the experiments use small generated graphs and short integer durations. The observed easy bound closure is evidence of successful certification, not evidence that state explosion has been solved. Future work should study structural parameterizations, industrial calendars, independently sourced benchmarks, and stronger relaxations for difficult residuals. An FPTAS or a universal $(2 - 1/(m + \varepsilon))$ claim requires separate complexity and approximation proofs and is not made here.

VII. CONCLUSION

BOS2L-DP turns a monotone-delay intuition into a precise fixed-prefix model, an exact finite-state solver, and explicit lower/upper formulas. Its central guarantees are (12) and (13), not an assumed perturbation of the original list-scheduling

ratio. The supplied experiments and counterexample separate unavoidable environmental deterioration from recoverable greedy-policy loss.

APPENDIX A INDEPENDENT MILP FORMULATION

Let $T = \{\tau, \dots, H - 1\}$, where H is a feasible heuristic completion. For unfinished task i , binary x_{it} means it executes in slot t . For initially unstarted tasks, binary y_{it} marks the first execution. The model minimizes C subject to

$$\begin{aligned} \sum_{t \in T} x_{it} &= w_i, & \sum_i x_{it} &\leq a(t), \\ x_{it} &\leq g_i(t), & C &\geq (t + 1)x_{it}, \end{aligned} \quad (21)$$

$$\begin{aligned} \sum_{t \in T} y_{it} &= 1, & y_{it} &\leq x_{it}, \\ x_{it} &\leq \sum_{s \leq t} y_{is} & (q_i^0 &= 0). \end{aligned} \quad (22)$$

For an edge into an initially unstarted v with positive threshold,

$$q_u^0 + \sum_{s < t} x_{us} \geq \ell_{uv} y_{vt}. \quad (23)$$

If u is already complete, its processing sum is absent. For a zero threshold and initially unstarted u ,

$$\sum_{s \leq t} y_{us} \geq y_{vt}. \quad (24)$$

Initially started tasks need no first-start variables; their historical handoff conditions have already been satisfied. Set $\tau \leq C \leq H$. These constraints encode exactly the same slot model while avoiding the DP's earliest-arrival recurrence. Returned schedules are independently replayed, and status, objective, dual bound, and gap are saved.

APPENDIX B ARTIFACT AND REPRODUCTION

The archive contains `src/dp_scheduler.py`, the independent `milp_oracle.py`, unchanged `legacy_bos_0915.py`, experiment and plotting scripts, regression tests, raw CSV, complete instance/schedule JSON certificates, this IEEEtran source, and a Traditional Chinese formula guide. Run

```
python -m unittest discover -s tests -v
python src/run_experiments.py
python src/stress_barriers.py
python src/make_figures.py
```

from the project root. Compile the manuscript with `latexmk -pdf` in `paper/`. Exact inputs and objectives are deterministic; timing measurements vary by machine. The local run used Python 3.12.14, NumPy 2.3.5, and SciPy 1.17.0. This manuscript is a research draft dated September 28, 2026; it has not undergone external peer review.

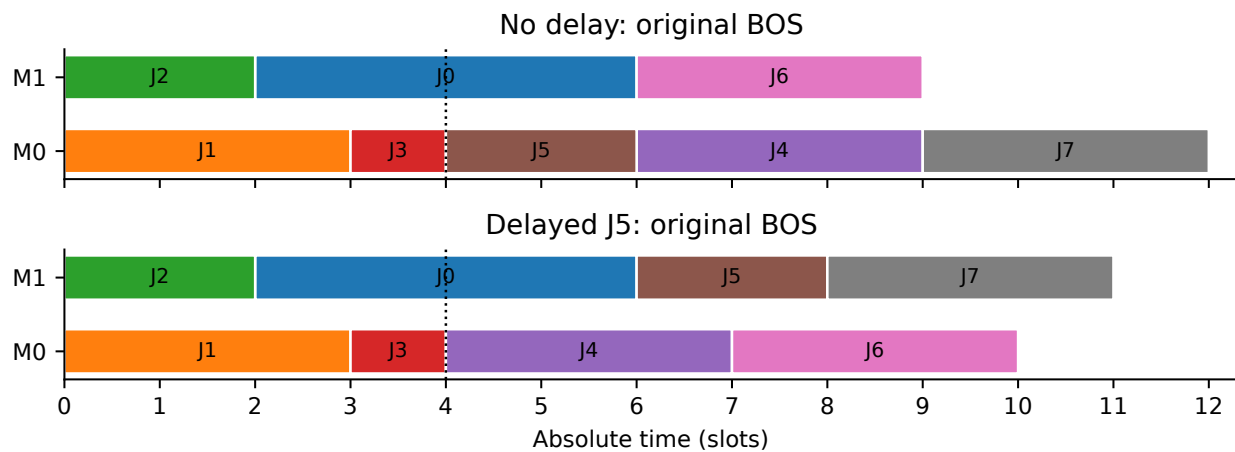


Fig. 3. Original September 15 BOS implementation: delaying task 5's release changes the greedy order and reduces makespan from 12 to 11. The dotted line marks the shared-prefix boundary $\tau = 4$. This is a policy anomaly, not an improvement of the optimum due to a disturbance.

REFERENCES

- [1] Chen-Hong, "BOS: Bottleneck-oriented overlap scheduling on identical parallel machines," 2026, unpublished manuscript and reproducible project, September 15, 2026.
- [2] R. L. Graham, "Bounds on multiprocessing timing anomalies," *SIAM Journal on Applied Mathematics*, vol. 17, no. 2, pp. 416–429, 1969.
- [3] R. Bellman, *Dynamic Programming*. Princeton University Press, 1957.
- [4] M. Bartusch, R. H. Möhring, and F. J. Radermacher, "Scheduling project networks with resource constraints and time windows," *Annals of Operations Research*, vol. 16, pp. 199–240, 1988.
- [5] S. Hartmann and D. Briskorn, "An updated survey of variants and extensions of the resource-constrained project scheduling problem," *European Journal of Operational Research*, vol. 297, no. 1, pp. 1–14, 2022.
- [6] P. Virtanen, R. Gommers, T. E. Oliphant *et al.*, "SciPy 1.0: Fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020.
- [7] SciPy developers, "scipy.optimize.milp reference," <https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.milp.html>, accessed September 28, 2026.